

Design Walk Through

Module Convenor: T. Nandha Kumaar

Email: nandhakumaar.t@nottingham.edu.my

**School of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus**



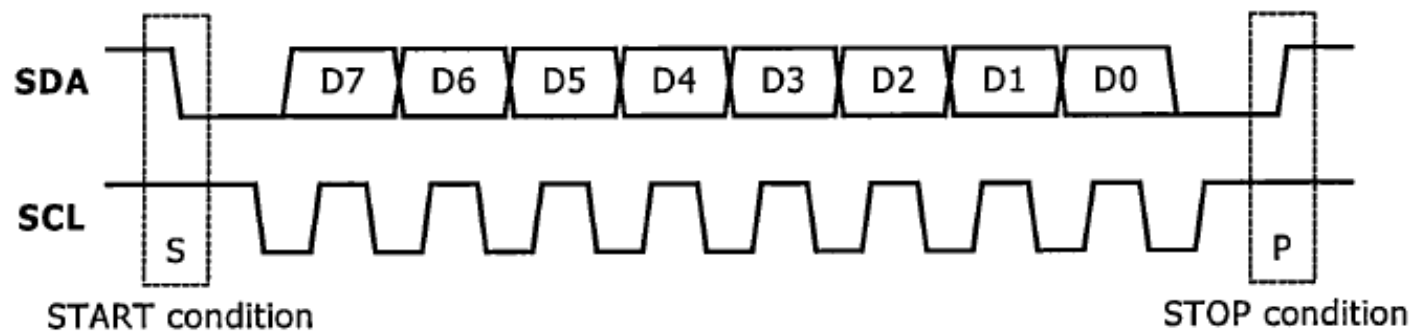
Topics to be covered

- I2C Bus
- Timer Watch
- RS232 Bus
- Type Converter

I²C Bus Description

Consider the I²C bus, which is widely used in electronic systems because only two bus lines, a serial data line (SDA) and a serial clock line (SCL), are required in the I²C bus.

Figure shows the basic I²C bus timing with two unique conditions: A HIGH to LOW transition on the SDA line while SCL is HIGH is the "START condition", a LOW to HIGH transition on the SDA line while SCL is HIGH is the "STOP condition". After the "START condition", 8-bit data will appear on the SDA line (one bit at a time), with the most significant bit coming first. The "STOP condition" indicates the completion of the I²C bus operation.



VHDL Design for I2C Bus

```
20 library IEEE;
21 use IEEE.STD_LOGIC_1164.ALL;
22 use IEEE.STD_LOGIC_ARITH.ALL;
23 use IEEE.STD_LOGIC_UNSIGNED.ALL;
24
25
26 entity I2cRxd is
27
28     port (Reset, SDA, SCL: in std_logic;
29           DataOut: out std_logic_vector (7 downto 0);
30           Ready: out std_logic);
31
32 end I2cRxd;
33
34 architecture Arch_I2cRxd of I2cRxd is
35
36     signal iStart, iStop, iEnable: std_logic;
37     signal iRegister: std_logic_vector (7 downto 0);
38     begin
39
40     process (Reset, SCL, SDA)
41         begin
42             if Reset = '1' then
43                 iStart <= '0';
44             elsif SDA'event and SDA = '0' then
45                 if SCL = '1' then
46                     iStart <= '1';
47                 end if;
48             end if;
49         end process;
50
```

```
51     process (Reset, SCL, SDA)
52
53     begin
54         if Reset = '1' then
55             iStop <= '0';
56         elsif SDA'event and SDA = '1' then
57             if SCL = '1' then
58                 iStop <= '1';
59             end if;
60         end if;
61     end process;
62
63     iEnable <= iStart xor iStop;
64
65     process (Reset, iEnable, SCL, SDA)
66     begin
67         if Reset = '1' then
68             iRegister <= (others=>'0');
69         elsif SCL'event and SCL = '0' then
70             if ienable = '1' then
71                 iRegister <= iRegister (6 downto 0) & SDA;
72             end if;
73         end if;
74     end process;
75
76     Ready <= iStop;
77     DataOut <= iRegister;
78 end Arch_I2cRxd;
```



Timer Watch

Consider a simplified watch with the following interface signals.

Signals	Directions	Descriptions
Reset	In	Reset the watch to zero hours, zero minutes and zero seconds
Clk	In	1-Hz clock input
Sec	Out	Output seconds
Min	Out	Output minutes
Hr	Out	Output hours

With the following entity declaration, use synthesisable VHDL to complete the VHDL design of the simplified watch.

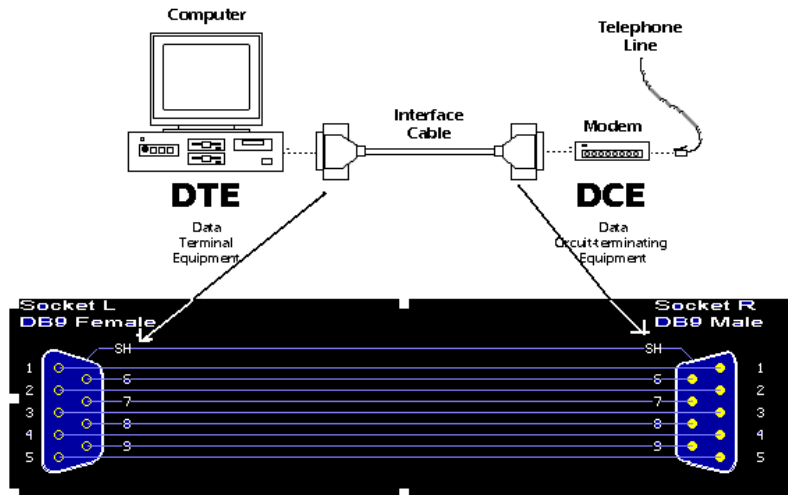
```
entity Watch is
  port(
    Reset, Clk: in std_logic;
    Sec, Min: out std_logic_vector (5 downto 0);
    Hr: out std_logic_vector (4 downto 0));
end Watch;
```

VHDL Design for Timer Watch

```
21 -- File: Watch.vhd
22 --
23 library ieee;
24 use ieee.std_logic_1164.all;
25 use ieee.std_logic_unsigned.all;
26
27 entity Watch is
28     Port(
29         Reset, Clk: in std_logic;
30         Sec, Min: out std_logic_vector (5 downto 0);
31         Hr: out std_logic_vector (4 downto 0)
32     );
33 end Watch;
34
35 architecture Arch_Watch of Watch is
36
37     signal iSecOut, iMinOut: std_logic;
38     signal iSec, iMin: std_logic_vector (5 downto 0);
39     signal iHr: std_logic_vector (4 downto 0);
40
41 begin
42
43     Process (Reset, Clk)
44     begin
45
46         if Reset = '1' then
47             iSec <= (others=>'0');
48             iSecOut <= '0';
49         elsif Clk'event and Clk = '1' then
50             if iSec = "111011" then
51                 iSec <= (others=>'0');
52                 iSecOut <= '1';
53             else
54                 iSec <= iSec + '1';
55                 iSecOut <= '0';
56             end if;
57         end if;
58
59     end process;
60
```

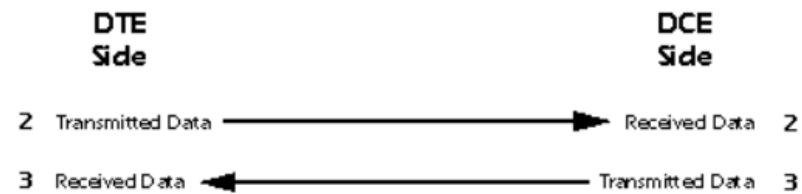
```
61
62     Process (Reset, iSecOut)
63     begin
64
65         if Reset = '1' then
66             iMin <= (others=>'0');
67             iMinOut <= '0';
68         elsif iSecOut'event and iSecOut = '1' then
69             if iMin = "111011" then
70                 iMin <= (others=>'0');
71                 iMinOut <= '1';
72             else
73                 iMin <= iMin + '1';
74                 iMinOut <= '0';
75             end if;
76         end if;
77     end process;
78
79     Process (Reset, iMinOut)
80     begin
81
82         if Reset = '1' then
83             iHr <= (others=>'0');
84         elsif iMinOut'event and iMinOut = '1' then
85             if iHr = "10111" then
86                 iHr <= (others=>'0');
87             else
88                 iHr <= iHr + '1';
89             end if;
90         end if;
91     end process;
92
93     Sec <= iSec;
94     Min <= iMin;
95     Hr <= iHr;
96
97 end Arch_Watch;
```

RS232 Bus



- RS232 is still one of the most widely used serial communication links, because of its low cost and reliability
- Used to connect between DTE (computer) and DCE (modem/LEDs) equipment.

- **Pin 1 - Received Line Signal Detector (CD):**
relevant when the DCE device is a modem.
- **Pin 2 - Transmitted Data (TxD):**
This signal is active when data is transmitted from the DTE device to the DCE device. When no data is transmitted, the signal is held in the mark condition (logic '1', negative voltage).
NOTE: Pin 2 on the DCE device is commonly labeled "Received Data".
- **Pin 3 - Received Data (RxD):**
This signal is active when the DTE device receives data from the DCE device. When no data is transmitted, the signal is held in the mark condition (logic '1', negative voltage).
NOTE: Pin 3 on the DCE device is commonly labeled "Transmitted Data".

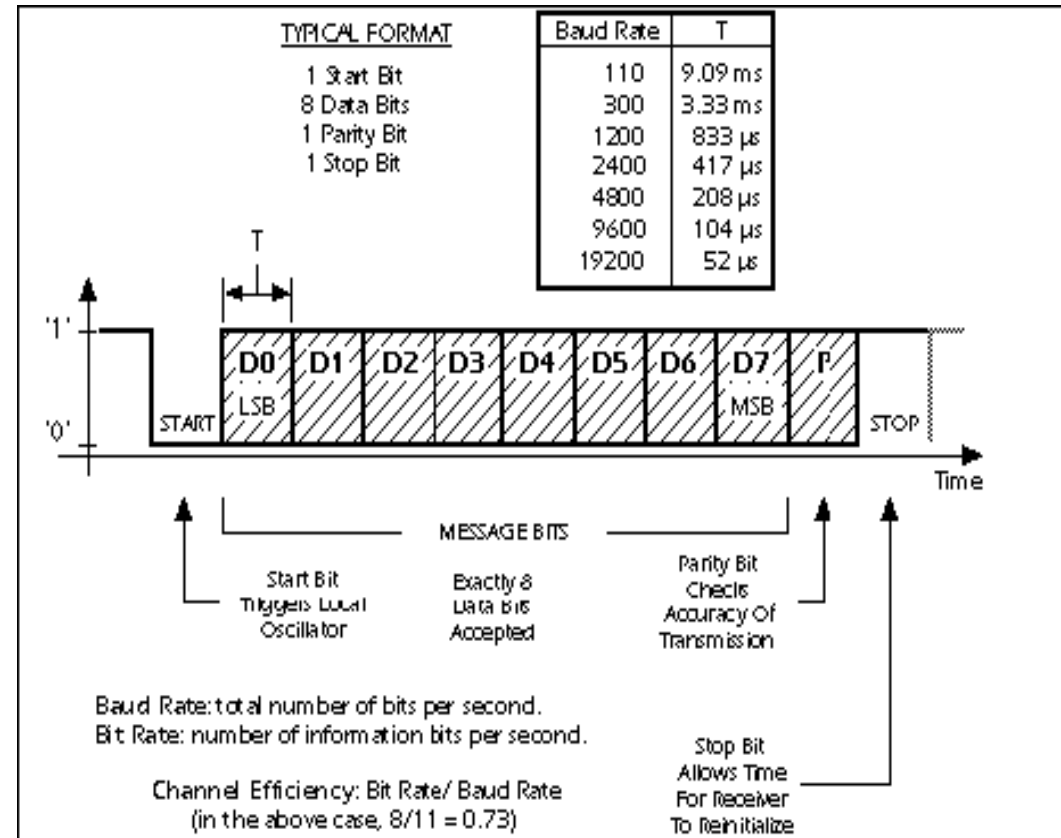




RS232 Bus

- **Pin 4 - DTE Ready (DTR):** This signal is asserted (logic '0', positive voltage) by the DTE device when it wishes to open a communications channel.
- **Pin 5 – Ground:** All signals are referenced to a common ground, as defined by the voltage on pin 5.
- **Pin 6 – DEC Ready:** When originating from a modem, this signal is asserted at logic '0', positive voltage.
- **Pin 7 - Request to Send (RTS):** This signal is asserted (logic '0', positive voltage) to prepare the DCE device for accepting transmitted data from the DTE device.
- **Pin 8 - Clear to Send (CTS):** This signal is asserted (logic '0', positive voltage) by the DCE device to inform the DTE device that transmission may begin.
- **Pin 9 - Ring Indicator (RI):** This signal is relevant when the DCE device is a modem.

- Most important signals
 - Transmitted Data (TxD)
 - Received Data (RxD)
- one start bit, 8 message bits, one parity bit and one stop bit
- parity bit
 - Can be disabled for a reliable communication link
- Baud Rate
 - directly proportional to channel's bandwidth
 - inversely proportional to the channel's noise levels,
 - specifies the number of RS232 data bits to be transmitted/received per second



Effective data rate (bytes/sec) = (Baud rate * Channel efficiency)/8

High Channel Efficiency = $(9600 * (8/10)) / 8 = 960$ bytes/sec [no parity bit]

RS232 Receiver VHDL Code



```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity Rs232Rxd is
6
7     port( Reset, Clock16x, Rxd: in std_logic;
8           DataOut1: out std_logic_vector (7 downto 0)
9         );
10 end Rs232Rxd;
11
12 architecture Rs232Rxd_Arch of Rs232Rxd is
13
14     attribute enum_encoding: string;
15
16     -- state definitions type stateType is (stIdle, stData, stStop, stRxdCompleted);
17
18     attribute enum_encoding of statetype: type is "00 01 11 10";
19
20     signal presState: stateType;
21     signal nextState: stateType;
22     signal iReset, iRxd1, iRxd2, iClock1xEnable, iClock1x, iEnableDataOut: std_logic ;
23     signal iClockDiv: std_logic_vector (3 downto 0) ;
24     signal iDataOut1, iShiftRegister: std_logic_vector (7 downto 0) ;
25     signal iNoBitsReceived: std_logic_vector (3 downto 0) ;
26
27 begin
```

```
29     process (Clock16x)
30     begin
31         if Clock16x'event and Clock16x = '1' then
32             if Reset = '1' or iReset = '1' then
33                 iRxd1 <= '1';
34                 iRxd2 <= '1';
35                 iClock1xEnable <= '0';
36                 iClockDiv <= (others=>'0');
37             else
38                 iRxd1 <= Rxd;
39                 iRxd2 <= iRxd1;
40             end if;
41             if iRxd1 = '0' and iRxd2 = '1' then
42                 iClock1xEnable <= '1';
43             elsif iClock1xEnable = '1' then
44                 iClockDiv <= iClockDiv + '1';
45             end if;
46         end if;
47     end process;
48
49     iClock1x <= iClockDiv(4);
50
51     process (iClock1xEnable, iClock1x)
52     begin
53         if iClock1xEnable = '0' then
54             iNoBitsReceived <= (others=>'0');
55             presState <= stIdle;
56         elsif iClock1x'event and iClock1x = '1' then
57             iNoBitsReceived <= iNoBitsReceived + '1';
58             presState <= nextState;
59         end if;
60         if iClock1x'event and iClock1x = '1' then
61             iShiftRegister <= Rxd & iShiftRegister(7 downto 1);
62             if iEnableDataOut = '1' then
63                 iDataOut1 <= iShiftRegister;
64             end if;
65         end if;
66     end process;
67
68     DataOut1 <= iDataOut1;
```

RS232 Receiver VHDL Code

```
70 process (presState, iClock1xEnable, iNoBitsReceived)
71 begin
72     -- signal defaults
73     iReset <= '0';
74     iEnableDataOut <= '0';
75     case presState is
76         when stIdle =>
77             if iClock1xEnable = '1' then
78                 nextState <= stData;
79             else
80                 nextState <= stIdle;
81             end if;
82
83         when stData =>
84             if iNoBitsReceived = "1001" then
85                 iEnableDataOut <= '1';
86                 nextState <= stStop;
87             else
88                 iEnableDataOut <= '0';
89                 nextState <= stData;
90             end if;
91
92         when stStop =>
93             nextState <= stRxdCompleted;
94
95         when stRxdCompleted =>
96             iReset <= '1';
97             nextState <= stIdle;
98     end case;
99 end process;
00 end Rs232Rxd_Arch;
01
```

- To achieve the high reliability for the RS232 reception,
- the sampling clock = 16 X baud rate = 153,600 Hz.
- The sampling frequency is generated from the on-board 50MHz clock by a clock divider with the divider factor of 325.
- For Simulation:
 - Disable the clock divider by disconnecting iClock16x from iCount9(8) and then routing System Clock signal directly to iClock16x signal in TopLevelRS232.vhd. Do **NOT** forget to enable the clock divider again after simulation.

Type Converter – Convert to ASCII

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- Uncomment the following lines to use the declarations that are
-- provided for instantiating Xilinx primitive components.
--library UNISIM;
--use UNISIM.VComponents.all;

entity conver2ascii is
  Port ( clk : in std_logic;
        reset : in std_logic;
        din : in std_logic_vector(7 downto 0);
                                ref : in std_logic; -- 0 - 1s, 1 - 10s
                                done : out std_logic;
        dout : out std_logic_vector(7 downto 0));
end conver2ascii;

architecture Behavioral of conver2ascii is

  signal counter : std_logic_vector(3 downto 0);
  signal result : std_logic_vector(3 downto 0);
  signal data : std_logic_vector(3 downto 0);
  signal rclk : std_logic;
  signal le : std_logic;
  signal zero : std_logic;
  signal stopstate : std_logic;
  signal dcount : std_logic_vector(7 downto 0);

  --*****
  -- STATE MACHINE SIGNAL DECLARATION:
  type StateType is ( Start, GetData, StartCount, stop );

  signal CurrentState, NextState : StateType;
  --*****
```



Type Converter – Convert to ASCII

```
begin
COMB: process(CurrentState, dcount)
begin
    case CurrentState is
        when start =>    NextState <= GetData;

        when GetData => NextState <= StartCount;

        when StartCount => if(dcount = 0) then
                            NextState <= stop;
                            else NextState <= StartCount;
                            end if;

        when stop =>    NextState <= stop;
    end case;
end process COMB;
SEQ: process(reset, clk)
begin
    if(reset = '0') then
        CurrentState <= Start;
    elsif (clk'event and clk = '1') then
        CurrentState <= NextState;
    end if;
end process SEQ;
with CurrentState select
    le <= '1' when GetData,
                                '0' when Others;

with CurrentState select
    stopstate <= '1' when Stop,
                                '0' when Others;

-- latch data and count down
process(clk, le)
begin
    if (clk'event and clk = '1') then
        if(le = '1') then
            dcount <= din;
        else
            dcount <= dcount - 1;
        end if;
    end if;
```



Type Converter – Convert to ASCII

```
        end if;
    end process;

    -- terminal counter
    process(clk, le, counter)
    begin
        if(le = '1') then
            counter <= (others => '0');
        elsif (clk'event and clk = '1') then
            if(zero = '1') then
                counter <= counter;
            elsif (counter >= 9) then
                counter <= (others => '0');
            else counter <= counter + 1;
            end if;
        end if;
    end process;

    -- result counter
    rclk <= '0' when (counter = 9) else '1';
    process(rclk, le)
    begin
        if(le = '1') then
            result <= (others => '0');
        elsif (rclk'event and rclk = '1') then
            if(zero = '1') then
                result <= result;
            else result <= result + 1;
            end if;
        end if;
    end process;

    zero <= '1' when ((dcount = 0) or (stopstate = '1')) else '0';
    done <= stopstate;
    data <= result when (ref = '1') else counter;
    dout <= "0011" & data;

end Behavioral;
```



END of Lecture 8